```
usb_device_unregister(old_child)
  -> usbhid remove()
  -> mark device disconnected
  -> usb_kill_urb()
       -> stop/mask the HID host channel
       -> wait for completion callback
  -> cancel_work_sync()
  -> input_unregister_device()
       -> remove event-minor registration
       -> wake blocked event readers
  -> release driver-owned references
```

注销 event 设备会唤醒阻塞在 `read()` 的客户端，剩下的语义与 7.4.7 小节一致：旧 fd 先读完已提交的完整帧，内存等最后一个引用释放。热插拔在这里第一次真正考验了这套引用计数——拔掉的瞬间 `evtest` 往往还活着，它必须看到断开错误而不是继续读到一个已经不属于自己的设备。

channel 7 怎么保证自己活着. Hub 的 interrupt-IN 是有周期要求的：`dwc2_hub_interrupt_start()` 提交 ch7 时按当前 `HFNUM` 的奇偶选择 `HCCHAR.ODDFRM`，把事务安排在下一个 USB frame；同时在 `dwc2_hub_wq` 上用 `mod_delayed_work()` 登记一个 `bInterval` 期限的看门狗。正常的 NAK 完成 IRQ 会把这条期限往后推；如果控制器复位后的第一笔事务根本没有产生完成 IRQ，期限到期就会重新启动 ch7。它只恢复 Hub 的 interrupt URB，不做任何 `GET_STATUS`，因此不是退回旧的端口轮询。新一代监控还会先等一个 `bInterval` 再首次提交，确保触发重枚举的那只旧 URB 的 completion 已经完全退出。下面这三行里后两行各只打印一次，合起来正好证明 delayed work 和 DWC2 host channel 两段链路都在运行：

```
dwc2: event-driven hub hotplug v2 scheduled on channel 7
dwc2: hub ch7 scheduler alive active=0 urb-state=0
dwc2: hub ch7 first IRQ hcint=... hctsiz=...
```

一次真机热插 port 3 键盘时，连接位图从 `0x2` 变为 `0xa`：`0x2` 表示 port 1 已连接，`0xa` 则表示 port 1 和 port 3 已连接，位图的 bit 号就是 root hub 的端口号，所以新增设备来自 port 3。Hub interrupt URB 通知 worker 后，可看到类似日志：

```
dwc2: hub interrupt-IN ep=1 mps=1 interval=12 ports=4
dwc2: hub IRQ change=8 connected=2->a; reprobe
dwc2: hub port 3 status=301 change=1
dwc2: hub port 3 after-reset status=303 speed=low
dwc2: hub port=3 child class=0 vid=2a7a pid=8a47 mps=8
input: USB HID Boot Keyboard registered as event0
usbkbd: IRQ-driven boot keyboard ready ep=1 mps=8 interval=10
```

`status=0x301` 表示设备已连接、端口已供电且设备为低速；reset 后的 `0x303` 还包含端口使能状态。若插入的是 Boot Mouse，USB core 会绑定鼠标驱动并注册相应 input 设备。枚举和 probe 日志只证明新设备已绑定、接收 URB 已提交；要证明实际按键路径恢复，还应看到 channel 6 的 completion 和首份有效 HID report。

拔出后的短暂 `ch6 error` 是正常窗口. 键盘刚拔掉、Hub 的状态端点还没把变化报上来之前，常见这几行：

```
dwc2: keyboard ch6 error intr=82 hctsiz=... hcsplt=...
usbkbd: channel 6 transfer error; rearming
```